

Basic Network Sniffer

A Python-based tool for capturing and analyzing live network traffic

Project Overview

This project is a lightweight network packet sniffer built in Python. It captures live network traffic and parses each packet to display key details such as source and destination IP addresses, transport-layer protocol (TCP / UDP / ICMP), port numbers, and payload contents. The goal of the project is to understand how data flows through a network at the packet level and how common protocols are structured.

Objectives

- Build a Python program to capture live network traffic packets.
- Analyze captured packets to understand their internal structure and content.
- Learn how data flows through a network and the fundamentals of networking protocols.
- Use libraries such as Scapy and the built-in socket module for packet capturing.
- Display useful information including source/destination IPs, protocols, and payloads.

Tools & Technologies Used

Component	Purpose
Python 3	Core programming language
Scapy	High-level packet capture and parsing
socket / struct (standard library)	Low-level raw-socket packet capture (Linux)
Npcap	Packet capture driver required on Windows

How It Works

- 1 The script opens a live capture session on a chosen network interface.
- 2 Every packet that passes through the interface is intercepted.
- 3 The IP layer is examined to extract the source IP, destination IP, TTL, and packet length.
- 4 The transport-layer protocol (TCP, UDP, or ICMP) is identified, and protocol-specific fields such as port numbers and TCP flags are extracted.
- 5 The payload (actual data being transmitted) is extracted and displayed in both hexadecimal and ASCII form.
- 6 All extracted information is printed to the console in a clean, timestamped, human-readable format.

Core Code Snippet (Scapy Version)

```

from scapy.all import sniff, IP, TCP, UDP, ICMP, Raw

def handle_packet(pkt):
    if not pkt.haslayer(IP):
        return
    ip_layer = pkt[IP]
    src, dst = ip_layer.src, ip_layer.dst
    if pkt.haslayer(TCP):
        tcp = pkt[TCP]
        print(f"TCP {src} -> {dst} "
              f"sport={tcp.sport} dport={tcp.dport}")
    elif pkt.haslayer(UDP):
        udp = pkt[UDP]
        print(f"UDP {src} -> {dst} "
              f"sport={udp.sport} dport={udp.dport}")

sniff(prn=handle_packet, store=False)

```

Sample Output

```

[12:10:51] TCP 192.168.100.24 -> 35.186.224.33 len=83 ttl=128 sport=33913 dport=443 flags=PA
    payload: hex[1703030026...] ascii[...&).x#.....a.3..V-.L...*...].]
[12:10:51] TCP 35.186.224.33 -> 192.168.100.24 len=40 ttl=117 sport=443 dport=33913 flags=A

```

Key Learnings

- Gained a practical understanding of how the TCP/IP protocol stack encapsulates data (Ethernet → IP → TCP/UDP → Application data).
- Learned to differentiate between transport-layer protocols based on port numbers and header fields.
- Understood why HTTPS payloads appear as unreadable/encrypted data, while unencrypted protocols reveal readable text.
- Practiced using both a high-level library (Scapy) and low-level raw sockets to achieve the same goal.

Ethical Use Notice

This tool was built strictly for educational purposes, to understand networking fundamentals. It was tested only on a private, personally owned network. Capturing network traffic without explicit authorization is illegal in most jurisdictions and is strongly discouraged.